

Mocking in xUnit

Mocking is a unit testing technique used to **replace real dependencies with simulated (fake) objects** so that a unit of code can be tested **in isolation**.

xUnit itself is a **testing framework**, not a mocking framework. In real-world projects, xUnit is commonly used **together with mocking libraries** such as **Moq**, **NSubstitute**, or **FakeItEasy**.

Why Mocking Is Needed

In unit testing, you should test **only the logic of the unit under test**, not its dependencies.

Typical dependencies include:

- Databases
- External APIs
- File systems
- Email/SMS services
- Other services or repositories

Mocking helps you:

- Avoid slow or unreliable external systems
 - Control return values and behaviors
 - Test edge cases and failure scenarios
 - Achieve fast, deterministic tests
-

Simple Conceptual Example (No Framework Yet)

Assume this service:

```
public class OrderService
{
    private readonly IProductRepository _repository;

    public OrderService(IProductRepository repository)
```

```

{
    _repository = repository;
}

public bool CanPlaceOrder(int productId)
{
    var stock = _repository.GetStock(productId);
    return stock > 0;
}
}

```

Problem without mocking:

- IProductRepository might hit a database
- Database availability affects your unit test
- Test becomes slow and unreliable

Solution:

- Replace IProductRepository with a mock that returns controlled data

Mocking with xUnit (Using Moq – Conceptually)

[Fact]

```

public void CanPlaceOrder_ReturnsTrue_WhenStockIsAvailable()
{
    // Arrange
    var mockRepo = new Mock<IProductRepository>();
    mockRepo.Setup(r => r.GetStock(1)).Returns(10);

    var service = new OrderService(mockRepo.Object);
}

```

```
// Act

var result = service.CanPlaceOrder(1);


// Assert

Assert.True(result);
}
```

What is mocked here?

- IProductRepository
 - Database call is completely avoided
 - Behavior is fully controlled by the test
-

Common Use Cases for Mocking in xUnit

1. Mocking Database Repositories

Use case: Testing service logic without SQL Server / EF Core

```
mockRepo.Setup(r => r.GetById(It.IsAny<int>()))
    .Returns(new Product { Id = 1, Name = "Laptop" });
```

2. Mocking External APIs

Use case: Avoid calling real payment gateways or third-party APIs

```
mockPaymentService.Setup(p => p.ProcessPayment())
    .Returns(true);
```

3. Mocking Failure Scenarios

Use case: Validate exception handling and error paths

```
mockRepo.Setup(r => r.Save(It.IsAny<Order>()))
```

```
.Throws(new Exception("DB failure"));
```

4. Verifying Interactions (Behavior Testing)

Use case: Ensure a method was called

```
mockRepo.Verify(r => r.Save(It.IsAny<Order>()), Times.Once);
```

5. Mocking Asynchronous Methods

Use case: Services using async/await

```
mockRepo.Setup(r => r.GetStockAsync(1))  
    .ReturnsAsync(5);
```

What Should Be Mocked vs Not Mocked

Mock These

- Repositories
- External services
- Message queues
- HTTP clients

Do NOT Mock

- The class under test
 - Value objects / DTOs
 - Simple utility methods
-

Key Takeaways

- Mocking isolates the unit under test
- xUnit is the test runner; mocking libraries provide fake objects
- Mocking improves test speed, reliability, and clarity

- Essential for service-layer and business-logic testing